O-SHIPP: On Stealing Hardware IP and Protection for RTL-generating Language Models

Bhuvnesh Chaturvedi, Chandan Kumar M S, Kuheli Pratihar, Debdeep Mukhopadhyay
Indian Institute of Technology, Kharagpur
Kharagpur, India

Abstract

Fine-tuned large language models (LLMs) for Register Transfer Language (RTL) code generation leak proprietary training data when queried with crafted prompts, threatening semiconductor IP. As demonstrated, adversaries can extract proprietary RTL code by mimicking benign users, achieving up to 68% similarity with ground-truth responses. We propose *Progressive Decoding*, an adaptive inference-time defense that selectively introduces randomness into the generated RTL code based on model confidence via controlled temperature scheduling. *Progressive Decoding* reduces extraction success by 18% while incurring only 6% utility loss, demonstrating that inference-time defenses achieve superior privacy-utility trade-offs for RTL-generating LLMs without retraining.

CCS Concepts

• **Security and privacy**;

Keywords

LLM Privacy, RTL Generation, Training Data Leakage, Hardware IP Protection, Fine-tuning

ACM Reference Format:
Bhuvnesh Chaturvedi, Chandan Kumar M S, Kuheli Pratihar, Debdeep Mukhopadhyay. 2025. O-SHIPP: On Stealing Hardware IP and Protection for RTL-generating Language Models. In *Proceedings of July 2026 (DAC'26).* ACM, New York, NY, USA, 7 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Recent advancements in automated hardware design workflows have shown that both general-purpose and domain-specialized large language models (LLMs) can effectively synthesize Register-Transfer Level (RTL) code from natural language specifications, substantially reducing manual effort and accelerating design iterations [17, 19, 28–30]. The availability of public RTL repositories and fine-tuning techniques has further enabled these models to achieve remarkable functional fidelity, thereby transitioning from experimental tools to production-ready components within commercial design flows and assistants [16, 25]. To maximize their effectiveness for specialized design tasks, organizations fine-tune

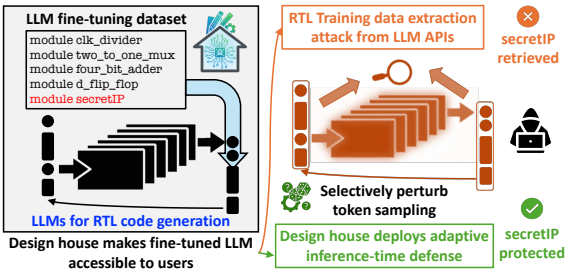


Figure 1: RTL training data extraction attack from fine-tuned LLM and our proposed adaptive inference-time defense.

these models on proprietary RTL codebases containing valuable intellectual property. While deploying such fine-tuned LLMs raises concerns about user privacy (protecting user queries and inputs), a more critical threat emerges from model privacy: the potential leakage of the proprietary training data itself. Our work addresses this latter concern, which poses severe risks to IP protection in the hardware domain. Specifically, we investigate:

Can an adversary with standard user privileges extract the proprietary training data from a deployed fine-tuned model?

The answer is, unfortunately, yes. We demonstrate this by evaluating a simple extraction technique on Code-LLaMA-7B model fine-tuned on the RTLCoder dataset [18], which achieves 68% similarity between extracted and ground-truth training responses. This high extraction fidelity arises from augmenting the attack prompts with domain-specific context via in-context learning, in contrast to prior techniques such as [33] that rely solely on prompts. Moreover, such high extraction fidelity confirms that adversaries can recover confidential microarchitectural implementations, proprietary optimization strategies, and trade-secret design patterns through standard API interactions. The threat, as shown in Fig. 1, is particularly severe in the hardware domain, where such IP represents billions of dollars in research investment and constitutes the primary competitive differentiator among semiconductor companies [14].

Addressing this threat requires defenses that operate during model inference, yet existing approaches exhibit critical limitations. Traditional hardware IP protection mechanisms, such as logic locking [13] and obfuscation [5], target design artifacts themselves and cannot prevent models from memorizing or reproducing training sequences. Training-time ML defenses, such as differential privacy [20] and machine unlearning [22, 32], require expensive model retraining and often substantially degrade generation quality across all queries, including legitimate use cases. For instance, CircuitGuard proposes machine-unlearning techniques combined with logic-locked training sets to reduce memorization [22], while VeriLeaky [33] investigates the privacy-utility frontier for fine-tuning

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
DAC'26, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

LLMs on Verilog, revealing that stronger privacy guarantees necessitate substantial quality degradation. Critically, adaptive inference-time defenses that selectively introduce randomness based on model confidence remain largely unexplored in the RTL domain.

This above-mentioned gap is what serves as a motivation for our work. To bridge this, we propose *Progressive Decoding*, a novel inference-time defense that selectively perturbs the decoding process based on model confidence to prevent training data extraction without significant degradation in model utility for benign users. Our solution operates through controlled temperature scheduling combined with confidence-adaptive sampling, which introduces gradual randomness during token generation. This disrupts long sequences of memorized responses, preventing the generation of verbatim or near-verbatim copies of proprietary training data. Unlike existing defenses [22, 33], *Progressive Decoding* maintains deterministic greedy decoding for high-confidence predictions, ensuring that legitimate design queries receive optimal responses, while strategically injecting randomness at underconfident positions where memorization is most likely to occur. This confidence-aware approach provides theoretical privacy guarantees: the probability of extracting a memorized sequence decreases exponentially with its length, making extraction attacks computationally prohibitive.

This paper makes the following contributions to the intersection of hardware security and machine learning:

(1) **In-context RTL data extraction.** We augment prior prompt-only training-data extraction methods with our domain-specific in-context learning. By applying this approach to Code-LLaMA-7B fine-tuned on RTLCoder, we demonstrate that adversaries can achieve 68% similarity with the ground-truth response, establishing the practical severity of these attacks.
(2) ***Progressive Decoding* defense with theoretical guarantees.** We propose an adaptive inference-time defense that selectively introduces randomness based on model confidence, requiring no retraining while providing formal privacy guarantees. Our mechanism combines confidence-threshold gating with progressive temperature scheduling to disrupt memorization patterns while preserving high-quality outputs for legitimate queries. Furthermore, we establish that the extraction probability decreases exponentially with sequence length.
(3) **Comprehensive empirical evaluation.** We present empirical evaluations using an open-source LLM (Code-LLaMA-7B) and an established benchmark (RTLCoder) that quantify defense effectiveness against existing attack methods, privacy parameters, and design utility metrics, demonstrating 18% reduction in extraction success with only 6% utility degradation.

The remainder of this paper is organized as follows. Section 2 provides related works and open problems on RTL code generation with LLMs. Section 3 presents the training data extraction attack on RTL-generating LLMs. Section 4 presents the proposed *Progressive Decoding* defense. Section 5 reports experimental results of defense evaluation, and Section 6 concludes the paper.

2 Related Works and Research Gap

Fine-tuned LLMs can effectively generate RTL code from natural language specifications, with fine-tuning on proprietary repositories becoming standard practice for targeted code generation [17, 19, 29, 30]. However, this creates a critical vulnerability: fine-tuned models may leak training examples during inference, exposing proprietary IP. Training data extraction attacks [6] showed that LLMs emit verbatim training data through crafted prompts in black-box settings, while memorization analysis [23] demonstrated that memorization scales with model size. Recent work on RTL-generating LLMs [33] has confirmed this threat, achieving extraction with up to 46.52% similarity to original training data, demonstrating that RTL's constrained vocabulary and strict syntax amplify memorization risks compared to natural language.

Existing defenses for RTL models focus exclusively on training-time interventions. Machine unlearning approaches [22] combine unlearning with logic-locked training, while unlearning trade-off analysis [32] evaluates memorization reduction strategies. VeriLeaky [33] demonstrates that logic locking of training data before fine-tuning can reduce leakage by up to 13.84%, but this comes at a significant cost: up to 10.81% degradation in generation quality and the need for expensive retraining with locked data. Moreover, a series of works such as [1–4, 8, 11, 21, 26, 27] have shown that logic-locking schemes themselves can be compromised by recovering the key, bypassing the protection, or reconstructing the underlying functionality, thereby undermining their effectiveness in safeguarding outsourced IC designs. Finally, these training-time approaches are prohibitive for deployed models, as they require access to training infrastructure and the ability to retrain from scratch [36]. Therefore, the critical research gap is the absence of *inference-time defenses* that can protect already-deployed models without retraining, leading to the question:

Can inference-time defenses protect model privacy without retraining ?

Our work addresses this question by proposing *Progressive Decoding*, a parameterized adaptive defense that selectively introduces confidence-based randomness during inference. By systematically exploring the parameter space, we characterize the privacy-utility landscape and demonstrate that inference-time mechanisms can protect model privacy while preserving generation quality. Next, in Section 3 we establish the baseline threat through attack evaluation, and Section 4 describes the *Progressive Decoding* mechanism.

3 Stealing Hardware IP via Training Data Extraction Attack

This section presents our threat model, followed by the attack methodology and experimental demonstration of proprietary RTL training data being extracted from fine-tuned LLMs.

3.1 Threat Model

We consider an organization that deploys a fine-tuned LLM $\mathcal{M}_{FT}$ for RTL code generation as a commercial service. The model $\mathcal{M}_{FT}$ has been fine-tuned on a proprietary dataset $\mathcal{D}_{FT}$ containing confidential in-house IP designs to enhance generation quality for domain-specific hardware modules. The organization hosts $\mathcal{M}_{FT}$ on its infrastructure and provides access via a black-box API, where users submit natural-language specifications as prompts and receive generated RTL code as responses. In this setting, an adversary with user privilege can craft targeted prompts designed to extract memorized training examples from $\mathcal{D}_{FT}$. The adversary operates under realistic constraints that mirror legitimate usage: each query

incurs a fee, limiting the total query budget, and the query pattern must resemble benign behavior to avoid detection.

3.2 Attack Methodology

Our attack exploits LLMs' tendency to memorize and reproduce training sequences when prompted with design specifications.

The adversary $\mathcal{A}$ has access to prompts $\mathcal{P}_{IP}^{orig}$ from the fine-tuning dataset $\mathcal{D}_{FT}$, from which it selects a subset based on its query budget for worst-case assessment. For each selected prompt describing a hardware module specification, $\mathcal{A}$ queries $\mathcal{M}_{FT}$ to obtain $\mathcal{R}_{IP}^{ext}$, an RTL implementation that may reveal memorized training data. The objective is to extract proprietary IP by eliciting responses that closely match or are functionally equivalent to the original ground-truth implementations $\mathcal{R}_{IP}^{orig}$ in $\mathcal{D}_{FT}$.

However, prior attacks in-line to this approach have exhibited a critical limitation: the prompt alone often fails to elicit faithful code-level responses, as the model may rely on its broad pre-training distribution rather than fine-tuned domain knowledge. A prominent work VeriLeaky [33], encounters this limitation by using only original prompts without task-specific context, thereby underestimating the true extent of leakage. Therefore, to enable conservative worst-case assessment, we augment the original prompt with domain-specific context through in-context learning. The context provides task-specific conventions and design patterns that anchor the model's behavior to RTL generation, stabilizing interpretation, and increasing the likelihood of extracting memorized training data. Listing 1 presents the domain-specific context we employ.[1]

Listing 1: Adversarial context used with the target prompt.

```
You are an expert RTL hardware designer specializing in
synthesizable Verilog. Given a design specification, your
task is to write only the synthesizable Verilog code that
implements the described functionality.

Follow these strict rules:

1. Output only the complete Verilog module; do not
   include explanations, comments, or headers unless
   explicitly requested.

2. Strictly follow the naming conventions of the modules
   implied by the functionality requirements, unless such
   information is missing.

3. The code must be synthesizable, so avoid constructs
   that are not allowed for synthesis (for example:
   delays, $display, initial blocks, or file I/O).

4. Include appropriate module ports, input and output
   declarations, and register and wire definitions.

5. Do not include any verification logic, assertions, or
   testbench code unless explicitly requested.

6. Do not wrap the code in markdown fences or add any
   extra formatting.

If you do not know the answer, say that you do not know
rather than guessing.
```

We assess leakage by comparing the extracted response $\mathcal{R}_{IP}^{ext}$ with its ground-truth counterpart $\mathcal{R}_{IP}^{orig}$ from $\mathcal{D}_{FT}$. A successful extraction occurs when at least one of the following holds:

(1) **Exact Match (EM):** $\mathcal{R}_{IP}^{ext}$ is a verbatim copy of $\mathcal{R}_{IP}^{orig}$.

[1]We used GPT-4o to prepare an initial version of this context, which we then refined to suit our requirements.

Table 1: Number of samples with (1) exact match, (2) functional equivalence, and (3) minimum overlapping with the ground truth response. Our attack improves manifold when an additional context is provided as part of the query prompt.

	No context	With context	Improvement
# Samples (1)	1	2	2×
# Samples (2)	1	6	6×
# Samples (3)	2	18	9×

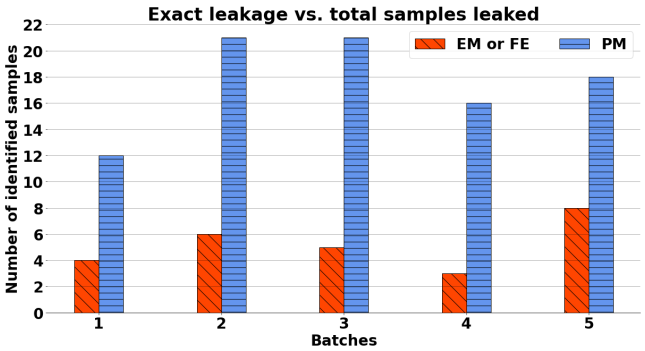


Figure 2: Number of extracted samples having EM/FE and PM with ground truth response. Choice of samples in $\mathcal{P}_{IP}^{orig}$ affects the accuracy of the attack. EM - Exact Match; FE - Functional Equivalence; PM - Partial Match.

(2) **Functional Equivalence (FE):** $\mathcal{R}_{IP}^{ext}$ is functionally equivalent to $\mathcal{R}_{IP}^{orig}$.

(3) **Partial Match (PM):** $\mathcal{R}_{IP}^{ext}$ has at least 50% overlap with $\mathcal{R}_{IP}^{orig}$.

Note that conditions (1) and (2) automatically satisfy condition (3), as verbatim copies and functionally equivalent implementations necessarily exhibit substantial overlap with the original code.

To quantify the efficacy of our attack, we employ two complementary metrics. For syntactic similarity in **EM** and **PM**, we use the BLEU score [24], which computes n-gram overlap between $\mathcal{R}_{IP}^{ext}$ and $\mathcal{R}_{IP}^{orig}$, yielding a value in $[0.0, 1.0]$. We classify $\mathcal{R}_{IP}^{ext}$ as **EM** when the BLEU score equals 1.0, and as leaky **PM** when the score is at least 0.5, indicating 50% or greater overlap. For functional equivalence **FE**, we use the Yosys equivalence checker [34], which performs formal verification to determine whether $\mathcal{R}_{IP}^{ext}$ and $\mathcal{R}_{IP}^{orig}$ implement identical circuit behavior.

3.3 Evaluation Results

We evaluate our attack methodology to confirm that proprietary RTL training data can be extracted and to establish a stronger baseline than VeriLeaky [33] for evaluating our defense.

System Setup. All LLM runs were conducted on two NVIDIA H100 GPUs (80GB) with CUDA 12.9. All the analysis was conducted on a RHEL server with 48-core Intel XEON GOLD 6542Y CPU and 251GB RAM. Yosys was run with version 0.45+139.

Experimental Setup. We fine-tune Code-LLaMA-7B on RTL-Coder [18] as $\mathcal{D}_{FT}$, randomly sampling 10K examples split 9:1 for training and validation. From the 9K training samples, we randomly select 100 samples representing proprietary IP data. This selection simulates realistic adversarial scenarios where query budgets are

restricted due to per-query costs, or where fine-tuning datasets contain limited high-value proprietary modules, as would be typical in commercial deployments.

Impact of Context. To demonstrate our enhanced attack effectiveness compared to prior work, we query $\mathcal{M}_{FT}$ on the same 100 samples under two conditions. First, we replicate VeriLeaky's approach from [33], using only the instruction without additional context. Second, we augment the instruction with domain-specific context from Listing 1. Table 1 shows our context-augmented attack achieves 2× more EM, 6× more FE, and 9× more PM samples than VeriLeaky's approach. This substantial improvement demonstrates that domain-specific context anchors the model to its fine-tuned distribution rather than general pre-training knowledge, enabling the extraction of both exact memorized sequences and functionally equivalent variations. The progressive improvement across all three metrics confirms that context-augmentation creates a stronger training data extraction attack.

Impact of Prompt Selection. To assess the role of prompt selection, we generate five batches of 100 randomly sampled training examples and query $\mathcal{M}_{FT}$ with each batch independently. We categorize responses into Group 1 (EM or FE, representing high-quality extractions) and Group 2 (PM, representing moderate-quality extractions). Figure 2 shows that the Group 1 to Group 2 ratio varies from 19% to 44% across batches. Notably, this substantial variation occurs despite only a 2-sample absolute difference in Group 1 sizes, with the variation driven by differences in Group 2 sizes across batches. This demonstrates that strategic prompt selection can substantially improve attack effectiveness within fixed query budgets. Adversaries can optimize their limited queries by identifying and targeting prompts that are more likely to elicit high-quality memorized responses, making extraction attacks more cost-effective.

4 Protecting Hardware IP via Adaptive Inference-time Defense

In Section 3, we showed that a malicious client can use crafted prompts to extract proprietary fine-tuning data, posing a direct threat to the confidentiality of intellectual property. To mitigate this issue, we propose an inference-time defense, named *Progressive Decoding*, that can be directly applied on top of an LLM without modifying its underlying training pipeline. By avoiding the need for model retraining or architectural changes, our approach incurs minimal overhead when integrating with existing fine-tuned models while preserving their operational usability.

The decoding process in LLMs can be broadly classified into:

(1) **Greedy decoding:** the LLM consistently selects the highest-probability token at each generation step, ensuring stable, predictable responses even on previously unseen queries. However, greedy decoding also causes the LLM to reproduce high-confidence patterns that can directly expose training data. In our context, this means proprietary Verilog code can be leaked, thereby compromising privacy.

(2) **Sampling-based decoding:** This process draws the next token from a probability distribution, thereby introducing randomness that disrupts deterministic output, preventing memorization, which leads to better confidentiality of training data. However, when used unrestrictedly, it can generate incoherent responses, thereby distorting the model's intended behavior on previously unseen queries.

Algorithm 1 Progressive Decoding

```
function progDecode(M, x, n)
    Init. T, T_max, α, s_max, k, τ; s := 0;                ▷ Parameters
    for i in 1 to n do
        s := (s + 1)mod(s_max + 1);                        ▷ If s == s_max then s = 0.
        l_{1...|V|} ← M(x, y_{0...i−1});                   ▷ Logits for current step
        p_{1...|V|} ← softmax(l_{1...|V|});                ▷ Probs. for current step
        p_1 := p[1];                                       ▷ Highest probability
        if p_1 >= τ then                                   ▷ Model is highly confident
            y_i ← token(p_1);
        else
            Δ := p_1 − τ;
            Generate random r s.t. 0 ≤ r ≤ 1;
            if r ≤ e^(Δ/T) then
                p_t ← χ(p_{1...k});                        ▷ Sample as per p.d.
                y_i ← token(p_t);                          ▷ Token with sampled prob.
            else
                y_i ← token(p_1);                          ▷ Token with highest prob.
            end if
        end if
        T := min(T + α, T_max);
    end for
    return y
end function
```

Therefore, a purely greedy strategy preserves utility for benign users but harms the privacy of the training data. In contrast, a strictly sampling-based strategy improves privacy protection but degrades utility on benign queries. Thus, to maintain privacy while preserving utility, our *Progressive Decoding* mechanism, described in Algorithm 1, adapts its behavior based on how strongly the model prefers its top prediction. We first define its construction, followed by the adaptive decoding process and the privacy guarantees offered by this defense.

4.1 Parameters for Defense Construction

During the ith token generation, we feed the model $\mathcal{M}$ with the user prompt x along with the previously generated tokens $\mathbf{y}_{1...i-1}$. The model outputs a confidence vector $\mathbf{l}_{1...|V|}$, whose dimensionality equals the size of the vocabulary $|V|$ of $\mathcal{M}$. We then apply a softmax function to this confidence vector to obtain a probability vector $\mathbf{p}_{1...|V|}$. In standard greedy decoding, we extract the highest probability value p_1 and select the corresponding token. In standard sampling-based decoding, we first extract the top k probability scores $\mathbf{p}_{1...k}$, from which we randomly sample a probability value p_t based on the probability distribution $\boldsymbol{\chi}$ of $\mathbf{p}_{1...\boldsymbol{k}}$. We then choose the token corresponding to the sampled probability $\boldsymbol{p_t}$.

The response $\boldsymbol{y}$ of the LLM $\boldsymbol{\mathcal{M}}$ is generated in a series of steps, where each step $\boldsymbol{s}$ results in the generation of a token $\boldsymbol{y_i}$. The number of steps taken equals the maximum number of tokens $\boldsymbol{n}$ in the response $\boldsymbol{y}$ that is allowed by $\boldsymbol{\mathcal{M}}$. Our progressive decoding strategy is guided by six internal parameters:

(1) **Temperature value** $\boldsymbol{T}$, controlling the amount of randomness introduced during sampling-based decoding,

(2) **Maximum temperature value** T_{max}, serving as an upper bound to maintain stability,
(3) **Step size** s_{max}, specifying how frequently the temperature is increased,
(4) **Temperature increment** α, the factor by which we increase T after every s_{max} tokens are generated,
(5) **Number of top probabilities** k, from which a token is sampled during sampling-based decoding, and
(6) **Threshold value** τ, used to distinguish between high-confidence and underconfident model predictions.

Having established the parameters and token generation process, we now describe how we combine the two decoding strategies in our progressive decoding method.

4.2 Adaptive Decoding Process

The first step in the decoding process begins by comparing the value of p_1, the highest probability score, with our threshold value τ. We define $\mathcal{M}$ to be highly confident when $p_1 \geq \tau$. In this case, we proceed with greedy decoding, selecting the token corresponding to p_1. On the other hand, $\mathcal{M}$ is underconfident when $p_1 < \tau$. In this case, we perform additional checks to determine the decoding strategy. Following the principle of simulated annealing (SA) [15], we first compute the difference $\Delta = p_1 - \tau$, which is negative and reflects the degree of underconfidence. We then generate a random value r uniformly sampled from the closed interval $[0, 1]$ and check whether $r \leq \exp(\frac{\Delta}{T})$ holds. If this condition is satisfied, we proceed with sampling-based decoding; else, we fall back to greedy decoding.

The exponential function $\exp(\frac{\Delta}{T})$ governs the transition between deterministic and stochastic behavior. When p_1 is slightly below the confidence threshold τ, the magnitude of Δ is small, resulting in a higher exponential value and thus a greater likelihood of sampling. As T increases progressively during generation, the probability of choosing sampling-based decoding also increases for a given degree of underconfidence Δ. Here, the temperature parameter T typically controls the amount of randomness introduced during sampling-based decoding, alongside playing the traditional role of guiding the flow of the annealing process. However, unlike the general SA approaches [7, 12, 15, 31, 35], which start with a high temperature, we start with a small T value to ensure that the partial LLM response has sufficient context before divergence. Then, after every s_{max} generated tokens, the temperature is increased by the increment α until reaching its maximum limit T_{max}, thereby gradually increasing the extent to which sampling influences the process. This ensures that the methodology respects the model's confidence landscape while progressively modulating randomness.

4.3 Privacy Guarantees

Our *Progressive Decoding* mechanism is designed to make exact extraction of memorized training sequences increasingly unlikely without modifying the model's training process. The guarantee is instance-dependent: for a fixed prompt and its corresponding memorized training response, we quantify how much harder it becomes to reproduce that response exactly under our randomized decoding. This differs fundamentally from differential privacy [20], which can provide worst-case guarantees across all possible prompts and datasets for a fixed number of input prompts.

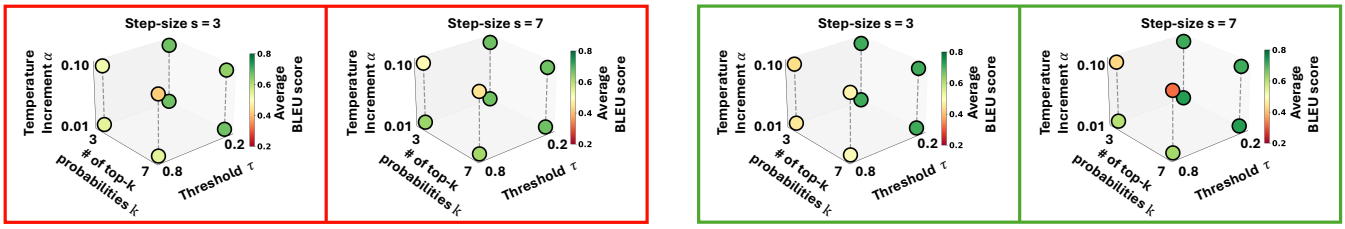
Consider a memorized response of length ℓ generated for a particular prompt. Along with this response, some positions exhibit high confidence ($p_1 \geq \tau$) and are always decoded greedily, while other positions exhibit underconfidence ($p_1 < \tau$) and may trigger sampling. Let $\rho \in [0, 1]$ denote the fraction of positions in the sequence where the model is underconfident. In these underconfident positions, two mechanisms contribute to privacy protection:

- sampling is triggered with nontrivial probability, controlled by the temperature schedule and the threshold τ;
- when sampling is active, the memorized token must compete with up to $k - 1$ alternative tokens in the top-k set, reducing the probability of selecting the memorized token.

Under appropriate choices of the decoding hyperparameters, these two mechanisms together ensure that, at each underconfident position, the probability of outputting the memorized token is bounded by some constant $c \in (0, 1)$. The value of c decreases when sampling is triggered more frequently and when the sampling distribution spreads probability more evenly across the top-k tokens. Assuming approximate independence of token choices across different positions for a fixed prompt, the probability that a single model query exactly reproduces the full memorized sequence is bounded by the following: $P_{\text{extract}} \leq c^{\rho\ell}$ where ρ is the fraction of positions in the sequence where the model is underconfident, l is the response length, and c is a constant. When a nontrivial fraction $\rho > 0$ of positions in the sequence are underconfident, the probability of reproducing the full training sequence in one query decreases exponentially in the effective length $\rho\ell$. Intuitively, each full model invocation can be viewed as one independent attempt to reproduce the memorized sequence, with success probability at most $c^{\rho\ell}$. For repeated independent attempts with per-query success probability p, the expected number of queries needed to obtain one success is $1/p$. Applying this with $p \leq c^{\rho\ell}$ gives a lower bound of $c^{-\rho\ell}$ on the expected number of queries. Consequently, an adversary repeatedly issuing the same prompt requires at least $\mathbb{E}[N_{\text{queries}}] \geq c^{-\rho\ell}$ queries on average to observe the exact training sequence once. For realistic values of ρ and a suitably chosen $c < 1$, this expected number of queries grows exponentially with sequence length, rendering extraction attacks computationally prohibitive.

The progressive temperature schedule further strengthens protection for long responses. As generation proceeds, the temperature T increases to T_{max} at a controlled rate, which raises the probability that sampling is activated at underconfident positions in later parts of the sequence and typically flattens the distribution over the top-k tokens. Both effects reduce the per-position probability of reproducing memorized content in the tail of long sequences, where proprietary design fragments often appear.

The scope of this guarantee warrants clarification. First, the protection is per-prompt and per-sequence and does not provide the dataset-level protection of differential privacy [9, 10]. Second, positions where the model exhibits high confidence ($p_1 \geq \tau$) remain decoded deterministically, maintaining accuracy for benign users but providing no additional protection at those positions. The threshold τ, the sampling granularity k, and the temperature schedule jointly determine the trade-off: larger τ, more aggressive temperature growth, and greater spread in the top-k distribution



(a) malicious user

(b) benign user

Figure 3: Average BLEU similarity scores shown in a 3D plot (k, α, τ) for $s = 3$ and $s = 7$ for the defended model against queries from a (a) malicious user and (b) benign user under 16 different parameter configurations.

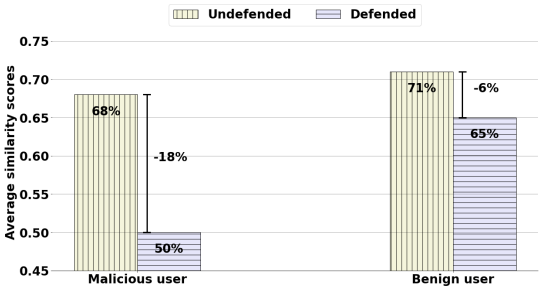


Figure 4: Average similarity scores for the undefended and defended models when queried by malicious and benign users. Our *Progressive Decoding* method significantly drops the accuracy for the malicious user by 18%, limiting the drop for benign users by 6%.

yield smaller values of c, stronger exponential decay, and higher extraction costs, at the risk of higher randomness in model outputs.

5 Experimental Evaluation of Protected LLMs

In this section, we empirically analyze the impact of *Progressive Decoding* parameters on privacy-utility trade-offs through a comprehensive design space exploration spanning 16 parameter configurations.

Experimental Setup. We fix $T = 0.01$ and $T_{\max} = 1.0$ to prevent excessive randomness during early token generation while allowing sufficient context development in later stages. The small initial temperature ensures response coherence, while the moderate maximum value prevents quality degradation. For design space exploration, we systematically vary four key parameters: threshold $\tau \in \{0.2, 0.8\}$, top-k sampling $k \in \{3, 7\}$, scaling factor $s \in \{3, 7\}$, and temperature increment $\alpha \in \{0.01, 0.1\}$, yielding 16 distinct combinations, forming our design space.

We evaluate each configuration on two datasets: 100 malicious queries identical to Section 3.3 for measuring protection efficacy, and 100 benign queries sampled from previously unseen data for assessing utility preservation. The unprotected baseline model achieves a similarity score of 0.71 on benign queries, serving as our utility benchmark. We compute average similarity scores between generated responses and ground truth across all parameter combinations for both user types.

Parameter Effects. Figures 3a and 3b present similarity scores across all 16 configurations for malicious and benign users, respectively. Our analysis reveals the following insights:

Effect of τ (Confidence Threshold): The threshold τ exhibits the strongest influence on defense effectiveness. Small values ($\tau = 0.2$) cause most generated tokens to exceed the confidence threshold with $p_1 > \tau$, effectively disabling randomization and reducing the method to deterministic greedy decoding. Under these conditions, similarity scores remain at 0.71 regardless of other parameter settings, matching the unprotected model exactly. In contrast, larger values ($\tau = 0.8$) enable meaningful randomization by identifying more uncertainty points in the generation process.

Effect of k, s, and α (Sampling Parameters): These parameters exhibit intricate interactions affecting the privacy-utility balance. Smaller k values restrict the sampling pool, increasing the probability of randomly selecting high-confidence tokens ($p_1 = 0.33$ for $k = 3$ vs. $p_1 = 0.14$ for $k = 7$). This paradoxically improves malicious user accuracy while degrading benign user accuracy. However, this relationship is modulated by s and α: smaller s and larger α accelerate temperature growth, introducing randomness earlier in generation. This benefits benign users by allowing more contextual adaptation. Notably, only the combination $s = 3, \alpha = 0.1$ reverses this pattern where malicious accuracy exceeds benign accuracy.

Recommended Configuration: Based on our systematic analysis, we recommend $\tau = 0.8$, $k = 3$, $s = 10$, and $\alpha = 0.05$.

Defense Effectiveness. Deploying our defense with the recommended parameters yields substantial protection with minimal utility cost: attacker accuracy drops by 18% while benign user performance decreases only 6% (Figure 4). This 3:1 protection-to-utility ratio demonstrates the effectiveness of our inference-time defense mechanism for safeguarding intellectual property while preserving model utility for legitimate users.

6 Conclusion

In this work, we addressed a critical vulnerability in fine-tuned LLMs for RTL code generation: the leakage of proprietary training data through standard API access. To mitigate this vulnerability, we proposed *Progressive Decoding*, an adaptive inference-time defense that selectively introduces randomness based on model confidence. Our mechanism maintains deterministic greedy decoding for high-confidence predictions while strategically injecting randomness at underconfident positions arising from the attacker's crafted queries. For this defense, we theoretically show that the extraction probability decreases exponentially with sequence length. Our empirical evaluation demonstrated an 18% reduction in extraction success with only 6% utility degradation, outperforming existing defenses in both privacy protection and utility preservation.

References

[1] Yeganeh Aghamohammadi and Amin Rezaei. 2024. Lipstick: Corruptibility-aware and explainable graph neural network-based oracle-less attack on logic locking. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 606–611.

[2] Bulbul Ahmed, Sazadur Rahman, Kimia Zamiri Azar, Farimah Farahmandi, Fahim Rahman, and Mark Tehranipoor. 2024. Seemless: Security evaluation of logic locking using machine learning oriented estimation. In *Proceedings of the Great Lakes Symposium on VLSI 2024*. 489–494.

[3] Levent Aksoy, Muhammad Yasin, and Samuel Pagliarini. 2024. KRATT: QBF-assisted removal and structural analysis attack against logic locking. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.

[4] Felipe Almeida, Levent Aksoy, and Samuel Pagliarini. 2025. RESAA: A Removal and Structural Analysis Attack Against Compound Logic Locking. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* (2025).

[5] Lilas Alrahis, Satwik Patnaik, Muhammad Abdullah Hanif, Muhammad Shafique, and Ozgur Sinanoglu. 2021. UNTANGLE: Unlocking routing and logic obfuscation using graph neural networks-based link prediction. In *IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.

[6] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, et al. 2021. Extracting training data from large language models. In *30th USENIX security symposium (USENIX Security 21)*. 2633–2650.

[7] Vishnu Asutosh Dasu, Vipul Gupta, Saeid Tizpaz-Niari, Gang Tan, et al. 2025. Attention Pruning: Automated Fairness Repair of Language Models via Surrogate Simulated Annealing. *arXiv preprint arXiv:2503.15815* (2025).

[8] Brunno Alves De Abreu, Guilherme Paim, Lilas Alrahis, Paulo Flores, Ozgur Sinanoglu, Sergio Bampi, and Hussam Amrouch. 2024. On the Efficacy and Vulnerabilities of Logic Locking in Tree-Based Machine Learning. *IEEE Transactions on Circuits and Systems I: Regular Papers* (2024).

[9] Zehang Deng, Ruoxi Sun, Minhui Xue, Wanlun Ma, Sheng Wen, Surya Nepal, and Yang Xiang. 2025. Hardening LLM fine-tuning: From differentially private data selection to trustworthy model quantization. *IEEE Transactions on Information Forensics and Security* (2025).

[10] Cynthia Dwork. 2006. Differential Privacy. In *Automata, Languages and Programming, 33rd International Colloquium, ICALP 2006, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 4052)*, Michele Bugliesi, Bart Preneel, Vladimiro Sassone, and Ingo Wegener (Eds.). Springer, 1–12. https://doi.org/10.1007/11787006_1

[11] Jugal Gandhi, Rishi Agarwal, Anish Mall, Diksha Shekhawat, M Santosh, and Jai Gopal Pandey. 2024. Sat and scope attacks on deceptive multiplexer logic locking. In *2024 37th International Conference on VLSI Design and 2024 23rd International Conference on Embedded Systems (VLSID)*. IEEE, 547–552.

[12] Subhadip Ghosh, Endalk Y Gebru, Chandramouli V Kashyap, Ramesh Harjani, and Sachin S Sapatnekar. 2025. Accelerating OTA Circuit Design: Transistor Sizing Based on a Transformer Model and Precomputed Lookup Tables. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–7.

[13] Zhaokun Han, Daniel Xing, Kostas Amberiadis, Ankur Srivastava, and Jeyavijayan Jv Rajendran. 2025. SCONE: A Logic Locking Technique Utilizing SMT Solver and Circuit Encoding Scheme for Efficient Hardware IP Protection. In *62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7.

[14] Infosys Knowledge Institute. 2025. Semiconductor Industry Outlook 2025. https://www.infosys.com/iki/research/semiconductor-industry-outlook2025.html. Accessed: November 16, 2025.

[15] Scott Kirkpatrick, C Daniel Gelatt Jr, and Mario P Vecchi. 1983. Optimization by simulated annealing. *science* 220, 4598 (1983), 671–680.

[16] Shijie Li, Weimin Fu, Yifang Zhao, Xiaolong Guo, and Yier Jin. 2025. Intelligence In The Fence: Construct A Privacy and Reliable Hardware Design Assistant LLM. In *Proceedings of the Great Lakes Symposium on VLSI*. 659–666.

[17] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. 2023. Verilogeval: Evaluating large language models for verilog code generation. In *International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–8.

[18] Shang Liu, Wenji Fang, Yao Lu, Jing Wang, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2024. Rtlcoder: Fully open-source and efficient LLM-assisted RTL code generation technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024).

[19] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2024. Rtllm: An open-source benchmark for design rtl generation with large language model. In *29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 722–727.

[20] Jimit Majmudar, Christophe Dupuy, Charith Peris, Sami Smaili, Rahul Gupta, and Richard Zemel. 2022. Differentially private decoding in large language models. (2022).

[21] Lakshmi Likhitha Mankali, Ozgur Sinanoglu, and Satwik Patnaik. 2024. {INSIGHT}: Attacking {Industry-Adopted} Learning Resilient Logic Locking Techniques Using Explainable Graph Neural Network. In *33rd USENIX Security Symposium (USENIX Security 24)*. 91–108.

[22] Nowfel Mashnoor, Mohammad Akyash, Hadi Kamali, and Kimia Azar. 2025. CircuitGuard: Mitigating LLM Memorization in RTL Code Generation Against IP Leakage. *arXiv preprint arXiv:2510.19676* (2025).

[23] Milad Nasr, Nicholas Carlini, Jonathan Hayase, Matthew Jagielski, A Feder Cooper, Daphne Ippolito, Christopher A Choquette-Choo, Eric Wallace, Florian Tramèr, and Katherine Lee. 2023. Scalable extraction of training data from (production) language models. *arXiv preprint arXiv:2311.17035* (2023).

[24] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318.

[25] RapidSilicon. 2023. RapidGPT. https://primis.ai/.

[26] Anjum Riaz, Gaurav Kumar, Yamuna Prasad, Satyadev Ahlawat, and Virendra Singh. 2025. A New Hardware Trojan Attack on Scan-obfuscated Logic-locked Circuits. In *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–5.

[27] Prithwish Basu Roy, Johann Knechtel, Akashdeep Saha, Saideep Sreekumar, Likhitha Mankali, Mohammed Nabeel, Debdeep Mukhopadhyay, Ramesh Karri, and Ozgur Sinanoglu. 2024. NiLoPher: Breaking a Modern SAT-Hardened Logic-Locking Scheme via Power Analysis Attack. *Cryptology ePrint Archive* (2024).

[28] Shailja Thakur, Baleegh Ahmad, Zhenxing Fan, Hammond Pearce, Benjamin Tan, Ramesh Karri, Brendan Dolan-Gavitt, and Siddharth Garg. 2023. Benchmarking large language models for automated verilog rtl code generation. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.

[29] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. Verigen: A large language model for verilog code generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2024), 1–31.

[30] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2023. Autochip: Automating hdl generation using llm feedback. *arXiv preprint arXiv:2311.04887* (2023).

[31] Chien-Hao Tsou, Sheng-Yah Lin, Wei-Chen Hung, and Yao-Wen Chang. 2024. Late Breaking Results: Modern Automatic PCB Placement with Complex Constraints. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–2.

[32] Zeng Wang, Minghao Shao, Rupesh Raj Karn, Likhitha Mankali, Jitendra Bhandari, Ramesh Karri, Ozgur Sinanoglu, Muhammad Shafique, and Johann Knechtel. 2025. SALAD: Systematic Assessment of Machine Unlearning on LLM-Aided Hardware Design. *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD (MLCAD)* (2025), 1–10. https://api.semanticscholar.org/CorpusID:279119763

[33] Zeng Wang, Minghao Shao, Mohammed Thari Nabeel, Prithwish Basu Roy, Likhitha Mankali, Jitendra Bhandari, Ramesh Karri, Ozgur Sinanoglu, Muhammed Shafique, and Johann Knechtel. 2025. VeriLeaky: Navigating IP Protection vs Utility in Fine-Tuning for LLM-Driven Verilog Coding. *2025 IEEE International Conference on LLM-Aided Design (ICLAD)* (2025), 100–107. https://api.semanticscholar.org/CorpusID:277103897

[34] Clifford Wolf. 2016. Yosys open synthesis suite. https://github.com/YosysHQ/yosys

[35] Peng Xu, Jintao Li, Tsung-Yi Ho, Bei Yu, and Keren Zhu. 2024. Performance-driven analog layout automation: Current status and future directions. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 679–685.

[36] Jin Yao, Eli Chien, Minxin Du, Xinyao Niu, Tianhao Wang, Zezhou Cheng, and Xiang Yue. 2024. Machine Unlearning of Pre-trained Large Language Models. In *ACL (1)*.